

vectra: Larger-than-Memory Tabular and Spatial Analysis in R

by Gilles Colling

Abstract Many analyses in ecology and other data-rich fields start from tables that do not fit in the memory of the computer running R, and end with a model or summary small enough to inspect. The **vectra** package connects the two within R. It combines a pull-based columnar query engine written in C, a compressed on-disk format with per-row-group statistics, streaming spatial operations built on GEOS, and interfaces that pass query results to R functions and model-fitting routines one batch or one partition at a time. This paper describes how the engine executes a query and states, for each class of operation, what it holds in memory and what it writes to disk. It then measures elapsed time, peak memory, and temporary disk use against **arrow**, **duckdb**, and **data.table** as workloads grow. A case study follows 401 million European vascular-plant occurrence records from GBIF through cleaning, a join to climate data, and per-region models of how geographical range size relates to climatic niche breadth.

1 Introduction

Occurrence records, vegetation plots, telemetry fixes, and remote-sensing extracts share a practical property: the raw table is large, the analytical result is small, and most of the work between them is filtering, joining, grouping, and reshaping. A European extract of vascular-plant records from the Global Biodiversity Information Facility (GBIF, [GBIF.org](https://gbif.org), 2026) holds about 400 million rows. Loaded as an R data frame with a dozen columns it would need more than 30 GB of memory, while the species-level table that a macroecological model uses has a few tens of thousands of rows. The question for the analyst is how to get from one to the other on the computer at hand.

R offers several answers, each covering part of this path. **data.table** (Barrett et al., 2026) makes in-memory manipulation fast and memory-frugal, but the table has to fit in memory. **arrow** (Richardson et al., 2026) and **duckdb** (Mühleisen et al., 2026; Raasveldt and Mühleisen, 2019) bring columnar query engines to R. Both accept **dplyr** verbs (Wickham et al., 2023), through **arrow**'s own bindings and through **dbplyr** (Wickham et al., 2024) or **duckplyr** (Mühleisen and Müller, 2025). Both read Parquet files (Apache Software Foundation, 2025) larger than memory, and DuckDB spills sorts, aggregations, and joins to disk. Earlier packages such as **ff** (Adler et al., 2024), **bigmemory** (Kane et al., 2013), and **disk.frame** (Dai, 2023) provide file-backed vectors and chunked data frames. **biglm** (Lumley, 2024) fits linear and generalised linear models in bounded memory from data supplied in chunks. For spatial data, **sf** (Pebesma, 2018) and **terra** (Hijmans, 2026) are the standard vector and raster tools. Both assume that the working layer, or at least the part of it being processed, is in memory.

The gaps sit at the joins between these tools. A query engine returns its result to R in one piece, so an analysis whose next step is an R function (a model fit, a custom summary, a per-group computation) either collects a result that may itself be too large, or writes glue code that pages through the result. Spatial operations on hundreds of millions of points leave the query engine for a spatial package that holds its input in memory. And the memory an analysis needs is rarely stated per operation, so users find out whether a pipeline fits by running it.

vectra (Colling, 2026c) addresses these gaps from inside R. It is an R extension with a query engine written in C11, reached through R's `.Call()` interface (R Core Team, 2026b). It provides **dplyr**-style verbs over a lazy query plan, a compressed columnar file format (`.vtr`), streaming readers for CSV, SQLite, and GeoTIFF, geometry operations on the GEOS C API (GEOS contributors, 2024) supplied by **libgeos** (Dunnington and Pebesma, 2025), and interfaces that hand a query result to R one batch (`collect_chunked()`, `chunk_feeder()`) or one on-disk partition (`offload()`, `group_map()`) at a time. The package has been on CRAN since 2026, and **taxify** (Colling, 2026a) depends on it.

This paper makes three contributions:

1. **A computational design.** It shows how lazy tabular queries, streaming spatial operations, and incremental or partitioned model fitting connect through three mechanisms: batches, disk-backed intermediate results, and partitions (Sections 2 and 3).
2. **An account of resource use.** For each class of operation it states what is held in memory, what spills to disk, and which quantity of the data (input rows, distinct groups, build-side rows, partition size, result size) sets the peak (Section 4).
3. **An evaluation and a worked analysis.** It measures these claims against **arrow**, **DuckDB**, and **data.table** on a machine with 8 GB of memory (Section 5). It then carries 401 million GBIF

records through to a fitted macroecological model and reports the scientific result alongside its computational cost (Section 6).

Section 7 discusses which workloads the measurements support using vectra for, where its semantics and resource bounds stop, and what its storage and dependency choices imply.

2 A working analysis with vectra

The example below is a condensed form of the case study in Section 6. It starts from a .vtr file of occurrence records, attaches a climate value to each record through a grid-cell key, reduces the records to one row per species and cell, and fits a model per European region. File names are shortened; the complete scripts are in the supplementary material.

```
library(vectra)

# 1. Open the records lazily. Nothing is read yet.
rec <- tbl("records.vtr")

# 2. Describe the cleaning step. Verbs build a plan; still nothing is read.
clean <- rec |>
  filter(occurrencestatus == "PRESENT", taxonrank == "SPECIES",
         is.na(unc) | unc <= 10000, year >= 1950) |>
  mutate(cell = floor((72 - lat) * 6) * 420 + floor((lon + 25) * 6)) |>
  select(specieskey, cell, datasetkey)

# 3. Execute: the plan runs batch by batch and the result streams to disk.
write_vtr(clean, "clean.vtr")

# 4. A second pipeline: one row per species and grid cell, joined to a
#     small table of cell attributes (climate, region, area).
tbl("clean.vtr") |>
  group_by(specieskey, cell) |>
  summarise(n = n(), n_ds = n_distinct(datasetkey)) |>
  inner_join(tbl("cells.vtr"), by = "cell") |>
  write_vtr("occ_env.vtr")

# 5. Split the occupancy table on disk by region, then run arbitrary R on
#     each region's shard, which is read into memory on its own.
p <- offload(tbl("occ_env.vtr"), by = "region")
fits <- group_modify(p, function(d, region) fit_one(species_table(d)))
```

Three things about this code matter for what follows. First, the verbs do not compute anything. `tbl()` returns a `vectra_node`, a reference to a plan tree held in C, and every verb wraps that tree in a new node. Computation happens only at a *terminal* operation, which pulls data through the plan: `collect()` returns a data frame, `write_vtr()`, `write_csv()`, and `write_sqlite()` stream to a file, `collect_chunked()` folds an R function over the batches, `chunk_feeder()` exposes the result as a resettable generator, and `offload()` materialises the result to disk as a replayable node or a set of partitions. A node is consumed by the terminal operation that runs it, so a plan that will be read twice is either rebuilt or offloaded.

Second, `explain()` prints the optimised plan without running it, so the memory a pipeline needs can be read before it runs. Section 3 follows one plan in detail.

Third, the example never holds the record table in R. Steps 3 and 4 write their results to disk, and step 5 reads one region's shard at a time. The largest object in R memory is one shard of the occupancy table, which is orders of magnitude smaller than the record table.

3 Execution and storage design

Plans, batches, and pull-based execution

A vectra plan is a tree of nodes. Each node implements a `next_batch()` function and a destructor, following the iterator model of query evaluation (Graefe, 1994). A terminal operation calls `next_batch()`

on the root. The root calls its child, and so on down to a scan, which reads one row group from its source and returns it as a *batch*. The batch then passes back up the tree, transformed at each node. Batches are columnar. Each column is a typed array (64-bit integer, double, logical, or string) with a validity bitmap for missing values, and whole columns are processed at a time rather than one row at a time (Boncz et al., 2005). A filter does not copy the rows it keeps. It attaches a *selection vector* of row indices to the batch, which downstream nodes read through. The batch size is set by the source: the row-group size of a `.vtr` file (131,072 rows by default), 65,536 rows for CSV and SQLite scans, and 256 raster rows for GeoTIFF scans.

Expressions in `filter()`, `mutate()`, and `summarise()` are captured in R, serialised into a small expression tree, and evaluated in C. This covers arithmetic, comparison and logical operators, `%in%`, `if_else()`, `case_when()`, `coalesce()`, date-time components, string functions (including regular expressions and edit distances), and geometry predicates and measures (`st_area()`, `st_intersects()`, and others) computed with GEOS on a hex-encoded well-known-binary (WKB) geometry column. Expensive per-row operations such as regular expressions, string distances, and geometry predicates run in parallel with OpenMP once a batch passes a size threshold. A function the engine does not know is an error, not a silent fallback to R.

The verbs are S3 methods on `vecetra_node` for generics that `vecetra` defines itself. `vecetra` is not a dplyr backend and does not import dplyr. Column selection (`select()`, `rename()`, `relocate()`, `across()`) goes through `tidyselect` (Henry and Wickham, 2024b), and argument capture through `rlang` (Henry and Wickham, 2024a).

The optimiser

Before execution, two rewrites run over the plan. *Predicate pushdown* attaches a filter that sits directly above a `.vtr` scan to the scan itself. The scan can then skip whole row groups in three ways. It can use a hash index (a `.vtri` sidecar file) for equality and `%in%` predicates. It can binary-search a column recorded as sorted. And it can use per-row-group minimum, maximum, and null-count statistics, which are a form of zone map (Moerkotte, 1998). *Column pruning* walks the tree from the root and marks the columns each node needs, so that a `.vtr` scan decodes only those. The optimiser also inserts hidden projections for nested aggregate expressions such as `mean(x * y)`.

Both rewrites are local. Pushdown applies only to a filter immediately above a scan. A filter placed after a `mutate()`, or above a `bind_rows()` of several files, is evaluated row by row on every batch. Column pruning stops at a `mutate()`, because the projection it creates passes every existing column through. Section 5 measures what these limits cost.

Following one query

Consider a table of plot observations stored in ten row groups, sorted by year, and a small table of site attributes. The query keeps the observations from 2015 onward, attaches each site's region, and computes mean cover per region and species:

```
q <- tbl(f_obs) |>
  filter(year >= 2015) |>
  select(site, species, cover) |>
  left_join(tbl(f_site), by = "site") |>
  group_by(region, species) |>
  summarise(n = n(), mean_cover = mean(cover))
explain(q)

#> vecetra execution plan
#> Groups: region
#>
#> GroupAggNode [materializes, 2 keys]
#>   SortNode [materializes]
#>     JoinNode [build right + stream left, left, 1 keys]
#>       ProjectNode [streaming]
#>         FilterNode [streaming]
#>           ScanNode [streaming, 4/6 cols (pruned), predicate pushdown, tdc stats]
#>           ScanNode [streaming, 2 cols, tdc stats]
#>
#> Output columns (4):
#>   region <string>
```

```
#> species <string>
#> n <double>
#> mean_cover <double>
#>
#> <offload grade: streaming scan>
#> passes over data : 1 per consumption (lazy)
#> peak memory      : 0(one batch)
#> I/O cost         : 0(n) per pass
#> note             : plain query node; re-reading re-runs the upstream pipeline
```

Reading the plan from the leaves upwards shows where work and memory go. The observation scan reads 4 of its 6 columns (year for the filter and the three selected columns) and carries the pushed-down predicate. Because the file is sorted by year, the row-group statistics exclude row groups that end before 2015 without reading them (the annotation `tdc stats` marks a scan that has these statistics). The filter and projection stream: each batch passes through and is released. The join's right-hand (*build*) side, 200 sites, is read into a hash table before the first observation batch arrives. The observations are the *probe* side and stream past it. The grouped aggregation is preceded by a sort node that the engine inserts itself: every grouped `summarise()` sorts its input by the grouping keys and then aggregates one group at a time. This makes the number of groups irrelevant to memory, at the cost of sorting even when there are only a few groups (Section 4).

The .vtr format

A .vtr file is a sequence of row groups. Each column of each row group is stored as one self-describing compressed block, written by the `tdc` codec library (Colling, 2026b) that is vendored into the package. The file ends with an index of row groups that records each block's offset, size, and statistics (minimum, maximum, null count), followed by a 24-byte trailer holding a 64-bit digest of the file's bytes. A scan reads the index once, then reads only the blocks of the columns and row groups it needs. By default each block is byte-shuffled and LZ-compressed. With `compress = "small"` the writer trial-encodes each block with several models (delta, dictionary, sparse, floating-point prediction) and entropy coders and keeps the smallest. String columns are dictionary-encoded, so `collect()` creates each distinct string once rather than once per row. Dates, date-times with their time zone, and factors with their levels round trip through column annotations.

Stores can grow without being rewritten. `append_vtr()` adds row groups, or columns, after the existing data and rewrites only the index and trailer, so building a store by repeated appends costs time linear in the rows written. `delete_vtr()` records deleted rows in a sidecar tombstone file. `create_index()` builds a hash index with one entry per distinct key and row group. The index is stamped with the store's row count and digest, so an index left over from a replaced store is ignored rather than used to prune row groups that no longer match.

Where results go

Table 1 lists the ways a query result leaves the engine. The distinction that matters for memory is whether the result is materialised in R as a whole, in pieces, or not at all.

Table 1: Terminal operations and what each holds in R memory.

Interface	Result in R	Typical use
<code>collect()</code> , <code>collect_sf()</code> , <code>pull()</code>	whole result	small results: summaries, subsets
<code>write_vtr()</code> , <code>write_csv()</code> , <code>write_sqlite()</code> , <code>write_tiff()</code>	none	large results, format conversion
<code>collect_chunked(x, f, .init)</code>	one batch + accumulator	one-pass reductions: counts, $X'X$ and $X'y$
<code>chunk_feeder(source)</code>	one batch	iterative fits: <code>biglm::bigglm()</code>
<code>offload(x)</code>	none (spilled to .vtr)	replay a prepared query across passes
<code>offload(x, by = key)</code>	none (one .vtr run set per key)	localise within-group dependence
<code>group_map()</code> , <code>group_modify()</code>	one partition at a time	per-group models or custom code

`collect_chunked()` is a left fold: it calls `f(acc, chunk)` on each batch in turn, so any reduction with a bounded accumulator (a count, a histogram, sufficient statistics) runs in one pass. With a combine function it also folds a partition shard by shard and merges the results. `chunk_feeder()` follows the `data(reset)` protocol of `biglm::bigglm()`. The model can therefore drive the iteration

itself and re-read the data on each pass of iteratively reweighted least squares (McCullagh and Nelder, 1989; Miller, 1992). Because a plan is consumed when it runs, `chunk_feeder()` takes either a function that rebuilds the plan or an offloaded node. `offload(x)` writes the result of `x` to a temporary `.vtr` file once and returns a node that replays it, so each model pass reads the prepared columns instead of re-running the upstream joins. `offload(x, by = key)` writes one set of run files per key value, per value range, or per hash bucket. `group_map()` then reads each shard into a data frame and calls an arbitrary R function on it.

Spatial operations

`vecra` has no geometry type. A geometry travels as a hex-encoded WKB string in an ordinary column, and point data may instead be given as two coordinate columns. The streaming spatial verbs, `spatial_join()`, `spatial_filter()`, `spatial_clip()`, and `spatial_map()` among others, differ from the tabular verbs in one respect: they execute when called. Each pulls its input batch by batch, processes the batch, buffers the output in R, and writes it to temporary `.vtr` run files once the buffer passes one eighth of the memory budget. It then returns a lazy node over those files. For recognised predicates on planar coordinates, `spatial_join()` and `spatial_filter()` parse the resident right-hand layer once into GEOS prepared geometries indexed by an STR tree. Each batch is then tested in C, in parallel, without converting it to an `sf` object. Other predicates, and geographic coordinates with spherical geometry enabled, fall back to calling `sf::st_join()` on each decoded batch. When both layers are too large to hold in memory, `spatial_join(partition = grid(cellsize))` bins both layers to a regular grid and joins one cell at a time. Raster data enter as tables (`tbl_tiff()` yields one row per pixel), through point extraction (`tiff_extract_points()`), or through the tiled `.vec` raster format with strip-wise `zonal()`, `focal()`, `terrain()`, and `warp()`. The spatial verbs need `sf` for CRS handling and for the fallback path, so `sf` is a suggested dependency that becomes a requirement as soon as they are used.

4 Memory requirements of analytical operations

Every operation in a `vecra` plan belongs to one of six classes, according to what it must retain between batches (Table 2). All buffering subsystems derive their limit from a single budget, `vecra_mem()`. It is set with `options(vecra.memory = "4GB")` and defaults to half of the physical memory.

Table 2: Classes of operation by the state they retain between batches.

Class	Operations	Peak memory set by
Streaming	<code>scans</code> ; <code>filter()</code> , <code>select()</code> , <code>mutate()</code> , <code>slice_head()</code> ; <code>bind_rows()</code> ; <code>write_*</code> (); <code>collect_chunked()</code> ; <code>chunk_feeder()</code>	one batch (row-group size x columns)
Bounded by output parameter	<code>slice_min()</code> / <code>slice_max()</code> with <code>with_ties = FALSE</code>	<code>k</code> rows
Budgeted, spills to disk	<code>arrange()</code> ; grouped summarise() incl. <code>median()</code> and <code>n_distinct()</code> ; window functions; hash joins; interval and fuzzy joins; index builds	<code>vecra_mem()</code> for the buffered state, plus one batch
Partitioned	<code>offload(by =)</code> ; <code>group_map()</code> ; <code>spatial_dissolve()</code> ; <code>spatial_join(partition = grid())</code>	one shard (the largest partition), or the routing buffer
Resident comparison side	<code>spatial_join()</code> , <code>spatial_filter()</code> , <code>spatial_clip()</code> against an <code>sf</code> layer; <code>materialize()</code> blocks	the resident layer + one batch + output buffer (<code>vecra_mem()/8</code>)
Materialised in R	<code>collect()</code> , <code>collect_sf()</code> , <code>pull()</code> ; fallbacks: <code>slice_tail()</code> , <code>reframe()</code> , ties in <code>slice_min()</code>	the size of the result

Streaming operations retain nothing between batches, so their memory is one batch regardless of how many rows pass through. A pipeline built only from these, and ending in a file sink or a bounded fold, runs on any input that fits on disk.

Operations bounded by an output parameter. `slice_min(x, n = k, with_ties = FALSE)` keeps a heap of the `k` best rows, so its state grows with `k`, not with the input.

Budgeted operations must see all input before producing some output. In `vecra` they accumulate state up to `vecra_mem()` and then spill.

- `arrange()` is an external merge sort (Knuth, 1998). It accumulates rows in column builders until their estimated size passes the budget, sorts them, and writes a sorted run. At the end it merges the runs with a heap. The number of runs merged at once is capped (at most 64), and is sized from the measured row width so that the merge buffers stay within half the budget. More runs are merged in several passes.

- A grouped `summarise()` always goes through this sort, then aggregates one group at a time. Memory therefore does not depend on the number of groups, unlike a hash aggregation. `median()` and `n_distinct()` need every value of a group and spill a single large group to disk as well.
- Hash joins build the right-hand table in memory and stream the left. If the build side exceeds the budget, both sides are hash-partitioned into 64 run files and joined one partition at a time, as in a Grace hash join (Kitsuregawa et al., 1983; Shapiro, 1986). A partition that is still too large is split again with a different hash, up to three levels. A partition that cannot be split, because a single key value is too frequent, is joined by a block-nested loop over budget-sized blocks.
- Grouped window functions sort by group and row order and process one group at a time, so they hold the largest group. Ungrouped windows stream in one pass, after a sort when the function needs an ordering (`ranks`, `lead()`). Interval joins sort both sides and sweep them, holding only the intervals open at the current position.

In this class the budget, not the data, sets the peak. Two qualifications apply. The budget bounds the engine's *estimate* of its buffered state. The sort that follows accumulation also needs working space, so peak process memory can exceed the budget (Section 5 measures by how much). And operations that spill trade memory for disk traffic: a sort that spills reads and writes its input at least once more.

Partitioned operations make one pass that routes rows to per-key run files, then process one partition at a time. `offload(x, by = key)` first writes `x` to a temporary file. It then reads that file batch by batch and routes each batch's rows into per-key buffers held as R data frames, flushing them to run files when their combined size passes `vectra_mem() / 8`. The routing cost is therefore paid in R. It also produces one run file per key per flush, so a key with many values produces many small files. `group_map()` and `group_modify()` then load one shard at a time, and their peak is the largest shard as an R data frame.

Resident comparison sides. A spatial join against an `sf` layer keeps that layer, its prepared geometries, and its spatial index in memory, together with one input batch and the output buffer.

Materialisation in R. `collect()` returns the whole result as a data frame, and it is the only terminal operation whose memory grows with the result. It is safe for summaries and selective subsets. For results of roughly the input's size, a file sink or a fold is the bounded alternative. Its cost is not only the data. An R data frame stores each value in R's own representation (8 bytes per double, a pointer per string), so a collected result is usually larger than its compressed `.vtr` form. A few operations fall back to R and collect their input: `slice_tail()`, `reframe()`, `slice_min()` with `with_ties = TRUE` (the `dplyr` default), and `distinct()` with `.keep_all = TRUE` on a column subset.

These classes compose. The memory of a pipeline is the sum over its simultaneously active stateful nodes, plus one batch per streaming stage. Because the stateful nodes share one budget knob rather than one budget, a plan with a sort and a spilling join can hold up to twice the budget. `explain()` marks each node as streaming or materialising, which is enough to find the nodes that count.

5 Comparative evaluation

Claims and experiments

Each claim of Sections 3 and 4 is paired with an experiment in Table 3. All experiments were run on one machine with two virtual CPUs and 8 GB of memory without swap (Section 8). A table larger than about 6 GB cannot be held in R there at all, and every tool must work within the same limit.

Setup

Data. The tabular benchmarks use a synthetic table shaped like an occurrence dataset. It has 11 columns: a row identifier, a year, four integer keys with 10^3 , 10^5 , 10^6 , and 10^7 distinct values, a species name drawn from 10^4 strings, two numeric measurements, and two coordinates. The table has 10^8 rows, written in ten files of 10^7 rows in each tool's own prepared format. That is `.vtr` for `vectra` (4.2 GB in total), Parquet with arrow's default compression for arrow and DuckDB (5.1 GB), and `fst` (Klik, 2022) for `data.table` (5.5 GB). An input of N rows is the first $N/10^7$ files. `vectra` combines them with `bind_rows()`, arrow with `open_dataset()`, DuckDB with a list passed to `read_parquet()`, and `data.table` reads them with `read_fst()` and binds them with `rbindlist()`, reading only the columns a query uses. The spatial benchmark tags the table's coordinates, which are uniform over a box from 10°W to 30°E and 36°N to 70°N , with 53 country polygons from Natural Earth 1:10m clipped to the same box.

Table 3: Claims and the experiments that test them, with the subsection that reports each.

Claim	Experiment	Evidence
Streaming operations use memory independent of input size	scan, filter, and aggregate 10 to 100 million rows	Input size and streaming
Stateful operations are bounded by the budget, not by groups or build-side rows	grouping into 1,000 to 10 million groups; joins with 1,000 to 50 million build rows; full sort	Stateful operations
Collecting a result costs memory in proportion to its size; sinks do not	keep 0.1% to 50% of 50 million rows, then collect() or write to a file	Input size and streaming
Pushdown, zone maps, pruning, and indexes reduce the data read	paired queries on one 50-million-row store (vectra only)	Optimiser ablation
Offloading a prepared query reduces the cost of iterative fits	logistic regression by IRLS over a joined, derived table	Repeated passes for model fitting
Streaming spatial joins remain practical beyond in-memory sizes	point-in-polygon tagging of 1 to 50 million points	Spatial joins
The components compose into a complete analysis	401 million GBIF records to per-region models	Case study

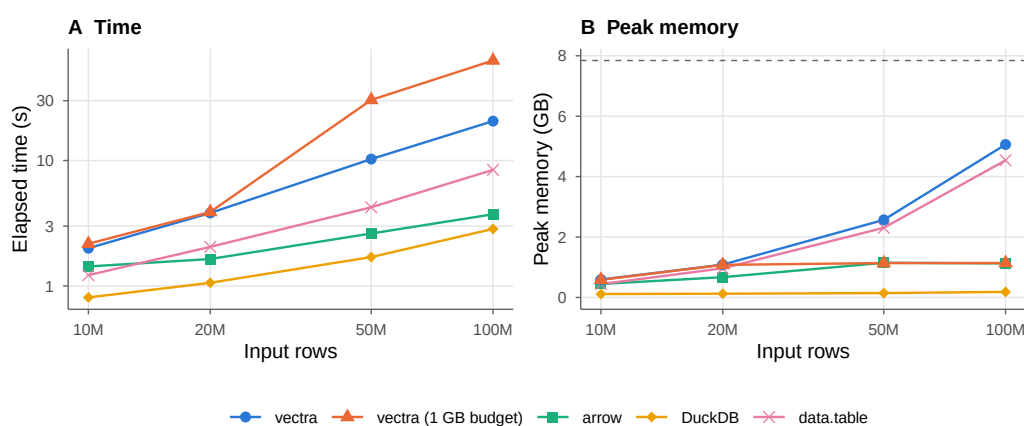


Figure 1: Scan, filter, derive, and aggregate by year (75 groups) as input grows from 10 to 100 million rows. (A) Elapsed time. (B) Peak resident memory of the R process. The dashed line is the machine’s physical memory.

Queries. Each workload was written the way a user of each package would write it: dplyr verbs for vectra and arrow, SQL for DuckDB, and `[.data.table]` syntax for data.table. No query was rewritten to suit one engine. Results that are large (grouped results, sorted tables) were written to each tool’s own file format, and small results were collected into R. For every configuration, the tools that completed returned the same checksum of their result (row counts and rounded sums).

Measurement. Every run was a fresh Rscript process started after the operating-system page cache had been dropped, so all reads were cold. Every tool was limited to two threads. Memory settings were left at each package’s defaults. For vectra that is a budget of half the physical memory (3.9 GB). A second configuration, “vectra (1 GB budget)”, sets `options(vectra.memory = "1GB")`. Elapsed time and peak resident memory were recorded by GNU time, which reports the maximum resident set size of the process and therefore includes every library’s native allocations. Temporary disk was the largest size of the process’s temporary directory, sampled every 0.5 seconds. Bytes read were taken from `/proc/self/io`. Each configuration was repeated 4 times, and figures show the median. A configuration whose first run was killed for lack of memory was not repeated, and the two slowest spatial runs were run once. Runs killed by the kernel’s out-of-memory handler are shown as crosses at the memory they had reached.

Input size and streaming

The first workload filters on two columns, derives a product, and aggregates by year, returning 75 rows (Figure 1). DuckDB was fastest at every size (2.9 s for 10^8 rows) and used the least memory (0.18 GB). arrow and data.table followed in time. data.table’s memory grew linearly with input, since it loads the three columns it needs, and reached 4.5 GB at 10^8 rows. vectra took 20.6 s at 10^8 rows. Its

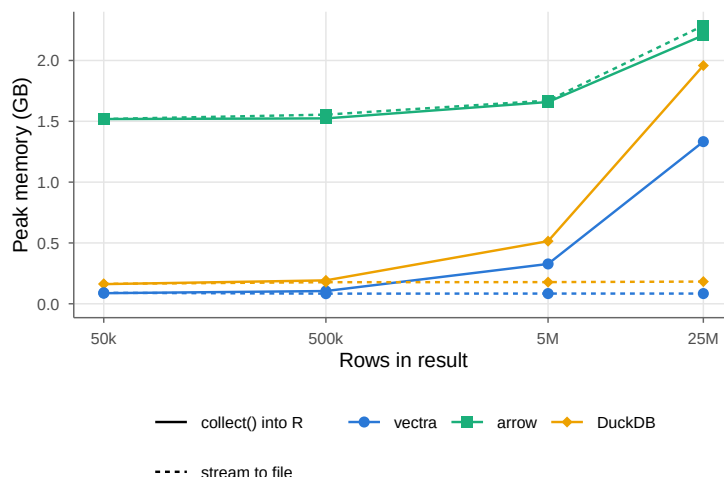


Figure 2: Memory needed to materialise a result in R. A filter keeps a fraction of 50 million rows and four columns; the result is either collected into R (solid) or written to a file (dashed).

memory also grew with input, to 5.1 GB, above its nominal budget. This is not a streaming cost. The grouped summarise() sorts its input, and because the filter is followed by a mutate(), column pruning stopped there and the sort carried all 12 columns of every surviving row (Section 3). With a 1 GB budget, vectra's memory stayed at 1.1 to 1.1 GB from 5×10^7 rows upwards. It spilled 1.2 GB to disk at 10^8 rows, and time rose to 62.8 s.

Pure streaming shows in Figure 2. A filter keeps between 0.1% and 50% of 5×10^7 rows and four columns. When the result was written to a file, vectra's peak memory stayed between 85 and 92 MB at every selectivity, the least of the three tools. DuckDB was similar (187 MB at most), and arrow used 2.3 GB. Collecting the result costs memory in proportion to its size for every tool. vectra needed 1.3 GB to return 25 million rows, arrow 2.2 GB, and DuckDB 2.0 GB. Writing the half-size result with vectra's default compress = "fast" was slower than collecting it (21.3 s against 7.2 s), because the encoder compresses every block.

Stateful operations

Grouping and joining (Figure 3) show the consequences of the sort-based design most clearly. vectra's memory for grouping did not depend on the number of groups (panel B). At the default budget it was 2.7 GB for 10^3 groups and 2.7 GB for 10^7 . With a 1 GB budget it was 1.9 GB at every group count, with 0.7 GB spilled. The hash-based engines grew with the number of groups: arrow from 1.1 to 3.1 GB, and DuckDB from 149 MB to 1.5 GB. Even at 10^7 groups, however, both stayed below vectra's default peak. The price of the sort is time. vectra took 9.3 s to aggregate into 10^3 groups, where DuckDB took 1.5 s and arrow 2.1 s. With a 1 GB budget, the extra read and write of the spill added about 25 to 30 s at every group count.

For joins, all three engines keep the build side in memory while it fits, and memory rose with its size (panel D). At 5×10^7 build rows, as many as the probe side, vectra at its default budget was killed for lack of memory at 5.8 GB. With a 1 GB budget it completed by partitioning both sides to disk, but its peak of 5.0 GB was five times the budget. The Grace partitioning bounds the hash table, but not the sort for the grouped aggregation that follows it. arrow was at the limit of the machine: it completed one of its three runs, at 5.8 GB, and was killed in the other two. DuckDB completed at 2.0 GB, and data.table at 2.7 GB.

Sorting (Table 4) is the operation where external memory is unavoidable once the table exceeds memory. At 10^7 rows, data.table and DuckDB sorted in about 5 s. vectra was the slowest tool at this size, and its in-memory path (51.6 s) was slower than its spilling path with a 1 GB budget (30.7 s). At 5×10^7 rows, which is about 4 GB of data in R, arrow and data.table were killed for lack of memory. DuckDB completed in 30.3 s with a peak of 5.3 GB. vectra took 158.7 s at either budget, and with a 1 GB budget its peak stayed at 1.2 GB. At 10^8 rows the two external sorts differed mainly in their disk use. DuckDB completed in 66.7 s, but it needed 7.2 GB of temporary files in addition to the 4.8-GB output. Its first two attempts failed because the 9 GB of free disk space then available was not enough, and the reported run was made after freeing space. vectra needed 4.3 GB of temporary files. It completed in 323.1 s with a peak of 1.2 GB under the 1 GB budget, and 4.4 GB under the default.

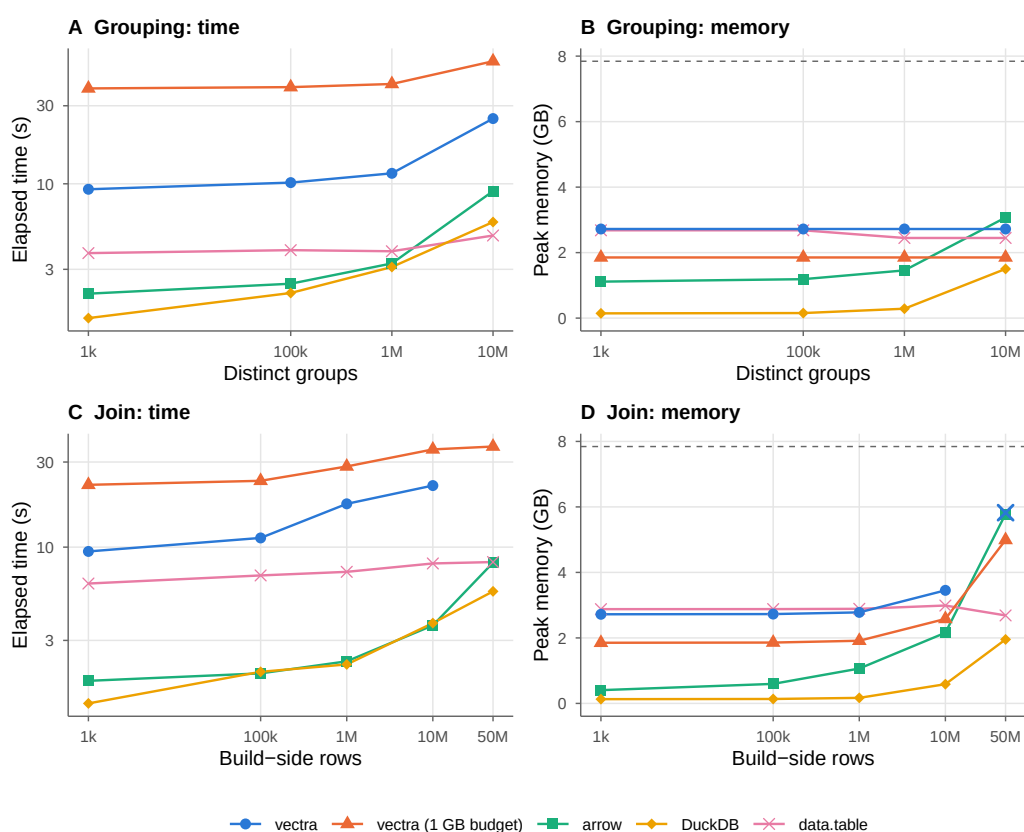


Figure 3: Grouping and joining 50 million rows. (A, B) Group by a key with 1,000 to 10 million distinct values and aggregate three columns; the result is written to a file. (C, D) Inner join to a build side of 1,000 to 50 million rows, then aggregate by year. Crosses mark runs killed for lack of memory.

Table 4: Sorting all 11 columns by a numeric column and writing the result to a file.

Rows	Tool	Time (s)	Peak memory (GB)	Temporary disk (GB)
10M	vectra	51.6	2.07	0.00
10M	vectra (1 GB budget)	30.7	1.17	0.42
10M	arrow	15.2	2.89	0.00
10M	DuckDB	5.0	1.47	0.00
10M	data.table	4.6	1.41	0.00
50M	vectra	158.7	4.35	2.12
50M	vectra (1 GB budget)	159.6	1.16	2.12
50M	arrow	killed (memory)	5.82	
50M	DuckDB	30.3	5.26	1.67
50M	data.table	killed (memory)	5.83	
100M	vectra	317.4	4.35	4.26
100M	vectra (1 GB budget)	323.1	1.17	4.25
100M	arrow	killed (memory)	5.82	
100M	DuckDB	66.7	5.55	7.25
100M	data.table	killed (memory)	5.83	

Optimiser ablation

Table 5: Paired queries on a single 50-million-row .vtr store (381 row groups). Each pair returns the same result. Data read is the number of bytes the process read from files.

Mechanism	Query	Time (s)	Data read (MB)
Zone maps	filter on a column ordered across row groups	0.42	4
	same selectivity (1%), column in random order	3.05	542
Column pruning	sum of 1 column	2.05	384
	sums of 10 columns	7.79	1863
Hash index	point lookup with a .vtri index	0.47	28
	same lookup without an index	6.20	2103
Pushdown order	filter() before mutate()	0.44	22
	the same filter after mutate()	7.65	2103

Table 5 isolates the optimiser’s mechanisms on a single store of 5×10^7 rows. Zone maps skipped almost all row groups for a filter on a column whose values are ordered across row groups. The query read 4 MB and took 0.42 s. The same selectivity on a randomly ordered column read every row group of the two columns involved (542 MB). Column pruning scaled the data read with the number of columns used. A .vtri index reduced a point lookup from 6.2 s and 2.1 GB read to 0.47 s and 28 MB. The last pair shows the cost of the optimiser’s locality. A filter written after a mutate() is not pushed into the scan, and the mutate() blocks pruning. The query then read the whole store and took 17 times as long as the same query with the verbs in the other order.

Repeated passes for model fitting

Table 6: Logistic regression on a prepared query (filter, join to a table of one million rows, derived response and predictors). Rows are fact-table rows before filtering. Passes are IRLS iterations to convergence.

Rows	Approach	Time (s)	of which preparation (s)	Passes	Peak memory (GB)	Data read (GB)
10M	collect(), then glm()	20.9	4.9	4	2.95	0.4
10M	bigglm(), plan rebuilt each pass	29.9		5	0.30	2.1
10M	offload() once, then bigglm()	15.1	7.5	5	0.16	0.9
50M	collect(), then glm()	killed (memory)			5.82	
50M	bigglm(), plan rebuilt each pass	142.6		5	0.29	10.3
50M	offload() once, then bigglm()	69.7	33.9	5	0.16	4.3

A generalised linear model fitted by iteratively reweighted least squares reads its data once per iteration. Table 6 fits a logistic regression to a query that filters the fact table, joins it to a table of 10^6 rows, and derives the response. Fed to biglm: :bigglm() through a plan factory, every iteration re-ran the scan and the join. For 5×10^7 fact rows the fit took 142.6 s and read 10.3 GB. Offloading the prepared query once took 33.9 s, after which each pass read the three prepared columns. The

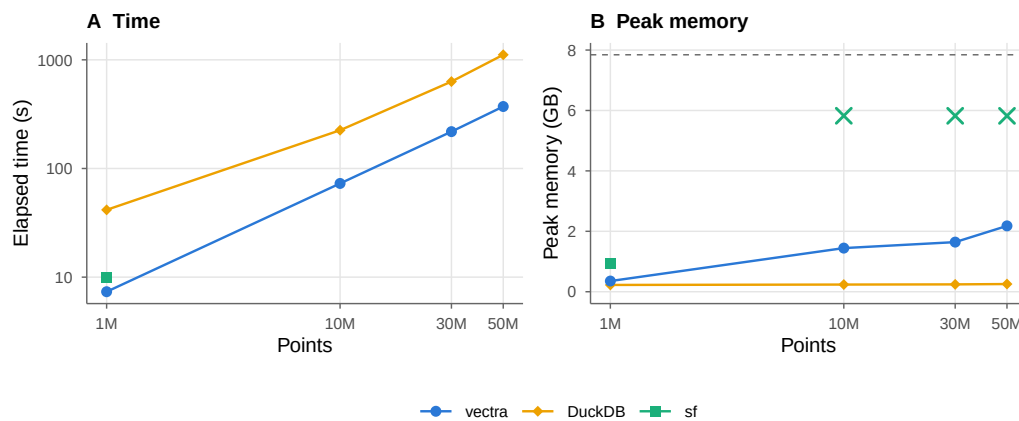


Figure 4: Tagging points with the European country that contains them (53 polygons) and counting points per country. vectra streams the points against GEOS prepared geometries; sf builds one sf object of all points and joins it in memory; DuckDB uses its spatial extension's spatial join. Crosses mark runs killed for lack of memory.

whole fit then took 69.7 s at 161 MB. Collecting the prepared data and calling `glm()` was a reasonable alternative at 10^7 rows, though it needed 2.9 GB. At 5×10^7 rows it was killed for lack of memory. All three approaches gave the same coefficients to ten decimal places.

Spatial joins

The spatial benchmark (Figure 4) tags each point with the country polygon containing it and counts points per country. At 10^6 points all three tools completed, vectra in 7.3 s, sf in 10.0 s, and DuckDB in 41.6 s. From 10^7 points upwards, building the sf point object and its join index exceeded memory, and the process was killed at about 6 GB. vectra's time grew linearly in the number of points, at about 135 thousand points per second on two threads, and it tagged 5×10^7 points in 6.2 minutes. Its peak memory rose from 1.4 to 2.2 GB. That rise comes from the output buffer, which holds joined rows as R data frames until they reach an eighth of the budget, and not from the input size. The spilled run files, which include a WKB point geometry for every output row, took 1.7 GB of temporary disk. DuckDB's spatial join used the least memory (261 MB) but was about three times slower (18.5 minutes for 5×10^7 points). Its plan indexes the polygons' bounding boxes and then tests each candidate pair with an unprepared exact predicate. vectra prepares each polygon once. At every size, all tools that completed matched the same number of points and found them in the same number of countries.

6 Case study: range size and climatic niche breadth of European plants

Question and data

Species with broad environmental tolerances are expected to occupy larger geographical ranges. This is the niche-breadth hypothesis for range size (Brown, 1984), and a meta-analysis found the relationship across most taxa and regions tested (Slatyer et al., 2013). Testing it on occurrence data requires, for every species, its range size and the breadth of climate it occupies. Both are computed from all of the species' records, which for European vascular plants means several hundred million rows. Both also increase with the number of records, because a species recorded more often is found in more cells and in more extreme climates. Sampling effort is well known to be uneven across GBIF data (Beck et al., 2014; Meyer et al., 2015), so it has to enter the model.

The records come from the GBIF occurrence snapshot of 1 September 2026 (GBIF.org, 2026), which GBIF distributes as Parquet files on public cloud storage. We selected the records of Tracheophyta whose country code is European, 401.0 million rows, and kept 12 columns: species key and name, coordinates, coordinate uncertainty, year, basis of record, occurrence status, taxon rank, country, and dataset key. Climate comes from WorldClim 2.1 (Fick and Hijmans, 2017) at 10 arc-minutes: annual mean temperature (BIO1) and annual precipitation (BIO12). Country and region come from the Natural Earth 1:10m country polygons (Natural Earth, 2024), grouped into the four European subregions of the United Nations geoscheme that Natural Earth records.

Workflow

The analysis has six stages, each a separate R script run in a fresh process (Table ??).

0. **Acquisition** (outside vectra). A DuckDB query over the snapshot's Parquet parts selected the rows and columns above and wrote them to 122 local Parquet files. In practice this step is a GBIF download request, for example through [rgbif](#) (Chamberlain et al., 2025).
1. **Ingest**. vectra does not read Parquet, so each file was read with arrow and appended to a single .vtr store with `append_vtr()`.
2. **Clean**. Records were kept if present, identified to species, collected in or after 1950, with a coordinate uncertainty of at most 10 km or none given, of an observation or specimen basis, and inside the climate grid. Each record was assigned the key of the 10-arc-minute cell containing it, computed from its coordinates in `mutate()`. The result streams to disk.
3. **Cells**. The two climate layers were read with `tbl_tiff()` as tables of pixels, given the same cell key, and joined. The cell centres were tagged with country and region by `spatial_join()` against the country polygons, dropping cells outside Europe. Each cell's area was computed from its latitude.
4. **Occupancy**. The cleaned records were reduced to one row per species and cell (the number of records, the number of datasets, and the first and last year), then joined to the cell table.
5. **Models**. The occupancy table was partitioned on disk by region with `offload(by = "region")`. `group_modify()` then read each region's shard into R, reduced it to one row per species, and fitted a linear model. A Europe-wide species table was computed in the engine with a grouped `summarise()` and modelled the same way.

For each species we computed range size as the summed area of occupied cells (km²). Niche breadth for temperature and for log precipitation is the standard deviation of the occupied cells' values. Within regions it was rarefied to 10 cells (the mean over 20 random draws of 10 occupied cells), so that breadth is measured at the same sample size for every species. The Europe-wide table uses the unrarefied standard deviation, which the engine computes in a grouped `summarise()`. We then fitted

$$\log(\text{range}) = \beta_0 + \beta_T \log(\text{breadth}_T) + \beta_P \log(\text{breadth}_P) + \beta_n \log(n_{\text{records}}) + \varepsilon$$

to species with at least 50 records in at least 10 cells. The coefficients β_T and β_P are elasticities: the proportional change in range size for a proportional change in niche breadth, at a given number of records.

Computational cost

TODO (pending run). Stage 0 finished for 44 of the 122 source groups before the session stopped. Stages 1 to 5 have been tested end-to-end on a 0.87-million-record sample but not yet run on the full extract. Fill Table ?? from `casestudy/stages.jsonl` and report the total time, peak memory per stage, and disk use.

Result

TODO (pending run). Report β_T and β_P by region with 95% intervals (the figure below), the number of species per region, and the comparison with the unrarefied Europe-wide model. Relate the sign and size of the elasticities to [Slatyer et al. \(2013\)](#).

7 Discussion

Workloads the measurements support

The evaluation separates two kinds of pipeline, and the case for vectra differs between them.

For **streaming pipelines**, which filter, derive, and select and then write to a file or fold the batches in R, vectra's peak memory stays near one batch regardless of input size. It was the lowest of the four tools on the collect and sink workloads, and it did not rise with input size (Section 5). These are the pipelines that convert, clean, and subset large sources before any modelling. They also connect to R most directly, through `collect_chunked()` and `chunk_feeder()`, without an intermediate copy of the result.

For **stateful relational work**, which groups, joins, and sorts, vectra completes these workloads within a configurable budget and spills beyond it. On this machine it was, however, consistently

slower than DuckDB and arrow, and at its default budget it used more memory than both (Section 5). The main cause is a design decision: every grouped aggregation passes through an external sort. That bounds memory independently of the number of groups, but it makes aggregation over a few groups cost as much as a sort of the aggregated columns. Where a pipeline is dominated by large aggregations or joins and needs to run many times, DuckDB is the faster engine. Where the table fits in memory, data.table remains the simplest fast option.

The case for vectra is therefore not raw relational speed. It is a single engine, installed as an ordinary R package, that covers the whole path from a source file too large for R, through grouping, joins, and point-in-polygon operations on the same stream, to a model fitted by R code on batches or partitions, with a memory account for every step. The spatial benchmark is where this composition pays most directly. vectra tagged points with countries faster than DuckDB's spatial extension and with a fraction of the memory sf needed, and sf could not complete the largest point sets at all.

Semantic limits

vectra implements dplyr-like verbs but is not a dplyr backend. Its generics mask dplyr's when both are attached, it covers a subset of the verbs, and it rejects expressions it cannot translate to its C evaluator rather than falling back to R. A few verbs (`slice_tail()`, `reframe()`, `slice_min()` with ties) do collect their input into R, and they are documented as doing so. Output order after grouping and joins is not guaranteed, as in SQL. A plan is consumed when it runs, which differs from arrow's and dbplyr's re-executable queries, and is the reason `offload()` and plan factories exist.

The optimiser is deliberately small. Predicate pushdown applies only to a filter placed directly above a `.vtr` scan, and column pruning stops at the first `mutate()`. The ablation in Section 5 shows that these limits change the cost of equivalent queries by more than an order of magnitude, so the order in which a user writes the verbs matters more than it does in engines with a cost-based optimiser. Pushing predicates through projections, and pruning through `mutate()`, are the next improvements to make.

Resource limits

The memory budget bounds each subsystem's estimate of its own buffered state. It is not a hard limit on the process. The measured peak of a grouped aggregation at the default budget exceeded the budget by the sorting working space, and a plan with several stateful nodes may hold several budgets at once. The spatial and partition verbs buffer their output as R data frames before spilling, so part of their memory is R's representation rather than the engine's. Partition routing in `offload(by =)` runs in R and writes one run file per key and flush, which becomes expensive for keys with many thousands of values. `method = "hash"` bounds the number of shards in that case. Spilling costs disk space as well as time: an external sort writes its input once more, and a spatial join on coordinate columns writes a WKB geometry column for every output row.

Storage and dependency choices

The `.vtr` format is specific to vectra. It gives the engine per-row-group statistics, dictionary-aware string decoding, in-place appends, fingerprinted indexes, and quantised or spatially predicted numeric columns, and it lets the package be built without any external compression library. The cost is interoperability. Other tools cannot read `.vtr`, and vectra does not read Parquet or Arrow IPC, the formats in which much large ecological data is now distributed, including the GBIF snapshots used in Section 6. In the case study, arrow read the Parquet files and vectra appended them to a `.vtr` store. The GeoTIFF reader decodes uncompressed and DEFLATE-compressed files but not LZW, the compression WorldClim distributes, so the climate layers were re-encoded once with **terra**. A Parquet reader, and LZW decoding for GeoTIFF, would remove both conversion steps.

"No external dependencies" would overstate the package's position. vectra's C code depends only on the C standard library and on sources it vendors, the **tdc** codec and the miniz deflate library. The readers and writers for CSV, SQLite database files, and GeoTIFF are vectra's own code. At the R level, however, the package imports **tidyselect**, **rlang**, **libgeos**, and **parallel**, and it links to **libgeos** for the GEOS C API. **libgeos** bundles GEOS, so no system library is needed, but it is a compiled dependency of about 3 MB. The spatial verbs additionally require **sf**, and with it GDAL and PROJ, although the geometry computations themselves run on the GEOS API without going through **sf**. The practical consequence is that the tabular engine compiles wherever R and a C11 compiler are available, while the spatial layer inherits **sf**'s installation requirements.

Summary

vecra lets an R user carry an analysis on data larger than memory from the source file to a fitted model within one package. It states, for each operation, which quantity of the data sets its memory. The measurements show streaming work at near-constant memory, stateful work bounded by a configurable budget at a clear cost in time relative to DuckDB, and streaming spatial joins that remain practical at sizes where in-memory spatial tools fail. The case study shows the pieces composing into a reproducible macroecological analysis of 401 million records on a machine with 8 GB of memory.

8 Computational details

The results in this paper were obtained with R 4.6.1 (R Core Team, 2026a) and vecra 0.12.4, arrow 25.0.1, duckdb 1.5.5, data.table 1.18.6.1, sf 1.1.3 (GEOS 3.12.1), and biglm 0.9.3, on Ubuntu 24.04 in a virtual machine with two virtual CPUs (Intel Xeon, 2.1 GHz), 8 GB of memory, no swap, and local SSD storage. The benchmark harness, the case-study scripts, and the raw measurements are in the supplementary material and in the package repository at <https://github.com/gcol33/vecra>. vecra is available from CRAN at <https://CRAN.R-project.org/package=vecra>.

9 Acknowledgements

We thank the data publishers who make their occurrence records available through GBIF.

Bibliography

- D. Adler, C. Gläser, O. Nenadic, J. Oehlschlägel, M. Schuemie, and W. Zucchini. *ff: Memory-Efficient Storage of Large Data on Disk and Fast Access Functions*, 2024. URL <https://CRAN.R-project.org/package=ff>. [p1]
- Apache Software Foundation. *Apache Parquet*, 2025. URL <https://parquet.apache.org/>. [p1]
- T. Barrett, M. Dowle, A. Srinivasan, J. Gorecki, M. Chirico, T. Hocking, B. Schwendinger, and I. Krylov. *data.table: Extension of 'data.frame'*, 2026. URL <https://CRAN.R-project.org/package=data.table>. R package version 1.18.6.1. [p1]
- J. Beck, M. Böller, A. Erhardt, and W. Schwanghart. Spatial bias in the GBIF database and its effect on modeling species' geographic distributions. *Ecological Informatics*, 19:10–15, 2014. doi: 10.1016/j.ecoinf.2013.11.002. [p11]
- P. Boncz, M. Zukowski, and N. Nes. MonetDB/X100: Hyper-pipelining query execution. In *Proceedings of the Second Biennial Conference on Innovative Data Systems Research (CIDR)*, pages 225–237, 2005. [p3]
- J. H. Brown. On the relationship between abundance and distribution of species. *The American Naturalist*, 124(2):255–279, 1984. doi: 10.1086/284267. [p11]
- S. Chamberlain, D. Oldoni, and J. Waller. *rgbif: Interface to the Global Biodiversity Information Facility API*, 2025. URL <https://CRAN.R-project.org/package=rgbif>. [p12]
- G. Colling. *taxify: Taxonomic Name Harmonisation*, 2026a. URL <https://CRAN.R-project.org/package=taxify>. R package version 0.5.0. [p1]
- G. Colling. *tdc: Typed Dimensional Compression*, 2026b. URL <https://github.com/gcol33/tdc>. C library, vendored into **vecra**. [p4]
- G. Colling. *vecra: Columnar Query Engine for Larger-than-RAM Data*, 2026c. URL <https://CRAN.R-project.org/package=vecra>. R package version 0.12.4. [p1]
- Z. Dai. *disk.frame: Larger-than-RAM Disk-Based Data Manipulation Framework*, 2023. URL <https://CRAN.R-project.org/package=disk.frame>. [p1]
- D. Dunnington and E. Pebesma. *libgeos: Open Source Geometry Engine ('GEOS') C API*, 2025. URL <https://CRAN.R-project.org/package=libgeos>. [p1]
- S. E. Fick and R. J. Hijmans. WorldClim 2: New 1-km spatial resolution climate surfaces for global land areas. *International Journal of Climatology*, 37(12):4302–4315, 2017. doi: 10.1002/joc.5086. [p11]

- GBIF.org. GBIF occurrence download (monthly snapshot of 1 september 2026), 2026. Accessed from the GBIF open-data bucket on Amazon Web Services. [p1, 11]
- GEOS contributors. *GEOS Computational Geometry Library*. Open Source Geospatial Foundation, 2024. URL <https://libgeos.org/>. [p1]
- G. Graefe. Volcano—an extensible and parallel query evaluation system. *IEEE Transactions on Knowledge and Data Engineering*, 6(1):120–135, 1994. doi: 10.1109/69.273032. [p2]
- L. Henry and H. Wickham. *rlang: Functions for Base Types and Core R and 'Tidyverse' Features*, 2024a. URL <https://CRAN.R-project.org/package=rlang>. [p3]
- L. Henry and H. Wickham. *tidyselect: Select from a Set of Strings*, 2024b. URL <https://CRAN.R-project.org/package=tidyselect>. [p3]
- R. J. Hijmans. *terra: Spatial Data Analysis*, 2026. URL <https://CRAN.R-project.org/package=terra>. [p1]
- M. J. Kane, J. W. Emerson, and S. Weston. Scalable strategies for computing with massive data. *Journal of Statistical Software*, 55(14):1–19, 2013. doi: 10.18637/jss.v055.i14. [p1]
- M. Kitsuregawa, H. Tanaka, and T. Moto-oka. Application of hash to data base machine and its architecture. *New Generation Computing*, 1(1):63–74, 1983. doi: 10.1007/BF03037022. [p6]
- M. Klik. *fst: Lightning Fast Serialization of Data Frames*, 2022. URL <https://CRAN.R-project.org/package=fst>. [p6]
- D. E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, 2 edition, 1998. [p5]
- T. Lumley. *biglm: Bounded Memory Linear and Generalized Linear Models*, 2024. URL <https://CRAN.R-project.org/package=biglm>. [p1]
- P. McCullagh and J. A. Nelder. *Generalized Linear Models*. Chapman and Hall, 2 edition, 1989. doi: 10.1007/978-1-4899-3242-6. [p5]
- C. Meyer, H. Kreft, R. Guralnick, and W. Jetz. Global priorities for an effective information basis of biodiversity distributions. *Nature Communications*, 6:8221, 2015. doi: 10.1038/ncomms9221. [p11]
- A. J. Miller. Algorithm AS 274: Least squares routines to supplement those of Gentleman. *Journal of the Royal Statistical Society. Series C (Applied Statistics)*, 41(2):458–478, 1992. doi: 10.2307/2347583. [p5]
- G. Moerkotte. Small materialized aggregates: A light weight index structure for data warehousing. In *Proceedings of the 24th International Conference on Very Large Data Bases*, pages 476–487, 1998. [p3]
- H. Mühleisen and K. Müller. *duckplyr: A DuckDB-Backed Version of 'dplyr'*, 2025. URL <https://CRAN.R-project.org/package=duckplyr>. [p1]
- H. Mühleisen, M. Raasveldt, and K. Müller. *duckdb: DBI Package for the DuckDB Database Management System*, 2026. URL <https://CRAN.R-project.org/package=duckdb>. R package version 1.5.5. [p1]
- Natural Earth. Natural earth: Free vector and raster map data at 1:10m, 1:50m, and 1:110m scales, 2024. URL <https://www.naturalearthdata.com/>. [p11]
- E. Pebesma. Simple features for R: Standardized support for spatial vector data. *The R Journal*, 10(1): 439–446, 2018. doi: 10.32614/RJ-2018-009. [p1]
- R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria, 2026a. URL <https://www.R-project.org/>. [p14]
- R Core Team. *Writing R Extensions*. R Foundation for Statistical Computing, 2026b. URL <https://cran.r-project.org/doc/manuals/r-release/R-exts.html>. [p1]
- M. Raasveldt and H. Mühleisen. DuckDB: An embeddable analytical database. In *Proceedings of the 2019 International Conference on Management of Data, SIGMOD '19*, pages 1981–1984. ACM, 2019. doi: 10.1145/3299869.3320212. [p1]
- N. Richardson, I. Cook, N. Crane, D. Dunnington, R. François, J. Keane, D. Moldovan-Grünfeld, J. Ooms, J. Wujciak-Jens, and Apache Arrow. *arrow: Integration to 'Apache' 'Arrow'*, 2026. URL <https://CRAN.R-project.org/package=arrow>. R package version 25.0.1. [p1]

- L. D. Shapiro. Join processing in database systems with large main memories. *ACM Transactions on Database Systems*, 11(3):239–264, 1986. doi: 10.1145/6314.6315. [p6]
- R. A. Slatyer, M. Hirst, and J. P. Sexton. Niche breadth predicts geographical range size: A general ecological pattern. *Ecology Letters*, 16(8):1104–1114, 2013. doi: 10.1111/ele.12140. [p11, 12]
- H. Wickham, R. François, L. Henry, K. Müller, and D. Vaughan. *dplyr: A Grammar of Data Manipulation*, 2023. URL <https://CRAN.R-project.org/package=dplyr>. [p1]
- H. Wickham, M. Girlich, and E. Ruiz. *dbplyr: A 'dplyr' Back End for Databases*, 2024. URL <https://CRAN.R-project.org/package=dbplyr>. [p1]

Gilles Colling
University of Vienna
Department of Botany and Biodiversity Research
Division of BioInvasions, Global Change & Macroecology
Rennweg 14, 1030 Vienna, Austria
<https://gillescolling.com>
ORCID: 0000-0003-3070-6066
gilles.colling051@gmail.com